

NED University of Engineering & Technology

Department of CIS Engineering



Batch: 2023

NM ASSIGNMENT

NAME: Fatima Shahid

ROLL NO: CS-23007

COURSE: Numerical Method (MT-442)

SUBMITTED TO: Hussain Kausar

Apply Numerical Methods using Python

(Graphical Analysis)

QUESTION

You are required to apply numerical methods using Python programming to solve the following problems and present the results graphically:

(a) Apply Numerical Integration Techniques:

Apply the following methods:

1. Trapezoidal Rule
2. Simpson's 1/3 Rule
3. Simpson's 3/8 Rule

To evaluate the integrals:

1. $\int_0^4 (x^2 + 2x + 1) dx;$ with $n = 4$
2. $\int_0^2 \ln(1 + x) dx;$ with $n = 6$
3. $\int_0^3 \frac{1}{1+x^2} dx;$ with $n = 6$

Apply plotting techniques to visualize each function and shade the area under the curve.

(b) Apply Numerical Methods for Differential Equations:

Apply:

1. Modified Euler Method
2. Runge-Kutta 4th Order (RK4) Method

to solve:

1. $\frac{dy}{dx} = x + y;$ $y(0) = 1$, upto $x = 0.2$, $h = 0.1$
2. $\frac{dy}{dx} = x^2 + y;$ $y(0) = 1$, upto $x = 0.2$, $h = 0.1$

Apply graphical comparison to plot:

Requirements:

1. Write complete Python code for all methods.

2. Plot graphs for:

- o Integration (area under curve)
- o Differential equations (numerical vs exact solution)

3. Display numerical results clearly in the output.

4. Provide a result statement explaining your findings and comparison of methods.

5. Use proper coding practices (functions, comments, structure).

6. Compile your work into a well-formatted PDF report that includes:

- o Python code
- o Graphs
- o Results and explanation

ANSWER

PART (a

CODE

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# Numerical Integration Methods
# -----

def trapezoidal_rule(f, a, b, n):
    h = (b - a) / n
    x = np.linspace(a, b, n+1) #It creates n+1 equally spaced points between a and b. since n intervals need n+1 points
    y = f(x)
    return (h/2) * (y[0] + 2*np.sum(y[1:n]) + y[n])

def simpson_1_3(f, a, b, n):
    if n % 2 != 0:
        raise ValueError("n must be even for Simpson 1/3")

    h = (b - a) / n
    x = np.linspace(a, b, n+1)
    y = f(x)
    return (h/3) * (y[0] + 4*np.sum(y[1:n:2]) + 2*np.sum(y[2:n-1:2]) + y[n])

def simpson_3_8(f, a, b, n):
    if n % 3 != 0:
        return "n must be multiple of 3 for Simpson 3/8"
    h = (b - a) / n
    x = np.linspace(a, b, n+1)
    y = f(x)
    return (3*h/8) * (y[0] + 3*np.sum(y[1:n][np.arange(1,n)%3!=0]) +
                    2*np.sum(y[3:n:3]) + y[n])

# -----
# Functions
# -----

f1 = lambda x: x**2 + 2*x + 1 # it acts as f1(x) where x**2 + 2*x + 1 is returned value
```

```

f2 = lambda x: np.log(1 + x)
f3 = lambda x: 1/(1 + x**2)

# -----
# Problem 1
# -----
a1, b1, n1 = 0, 4, 4

T1 = trapezoidal_rule(f1, a1, b1, n1)
# Replace the curve between each pair of points with a straight line
# Each segment forms a trapezoid works okay, but not very accurate for curves
S1_13 = simpson_1_3(f1, a1, b1, n1)
# A parabola follows curvature better than a straight line
# More accurate than trapezoidal n must be even (because it uses pairs of intervals)
# n is the number of intervals number of points n+1
S1_38 = simpson_3_8(f1, a1, b1, n1)

# -----
# Problem 2
# -----
a2, b2, n2 = 0, 2, 6

T2 = trapezoidal_rule(f2, a2, b2, n2)
S2_13 = simpson_1_3(f2, a2, b2, n2)
S2_38 = simpson_3_8(f2, a2, b2, n2)

# -----
# Problem 3
# -----
a3, b3, n3 = 0, 3, 6

T3 = trapezoidal_rule(f3, a3, b3, n3)
S3_13 = simpson_1_3(f3, a3, b3, n3)
S3_38 = simpson_3_8(f3, a3, b3, n3)

# -----
# Display Results
# -----
print("---- Integration Results ----")
print("(1)  $\int (x^2+2x+1) dx$  from 0 to 4")
print("Trapezoidal:", T1)
print("Simpson 1/3:", S1_13)

```

```

print("Simpson 3/8:", S1_38)

print("\n2)  $\int \ln(1+x) dx$  from 0 to 2")
print("Trapezoidal:", T2)
print("Simpson 1/3:", S2_13)
print("Simpson 3/8:", S2_38)

print("\n3)  $\int 1/(1+x^2) dx$  from 0 to 3")
print("Trapezoidal:", T3)
print("Simpson 1/3:", S3_13)
print("Simpson 3/8:", S3_38)

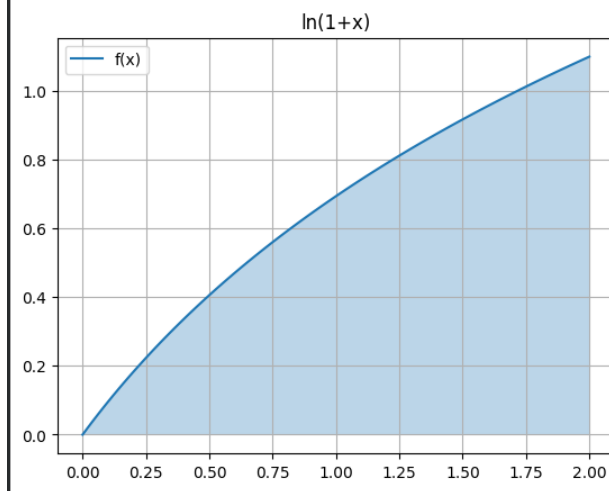
# -----
# Plotting (Area Shading)
# -----

def plot_area(f, a, b, title):
    x = np.linspace(a, b, 100)
    y = f(x)
    plt.figure()
    plt.plot(x, y, label="f(x)")
    plt.fill_between(x, y, alpha=0.3)
    plt.title(title)
    plt.legend()
    plt.grid()
    plt.show()

plot_area(f1, a1, b1, "x^2 + 2x + 1")
plot_area(f2, a2, b2, "ln(1+x)")
plot_area(f3, a3, b3, "1/(1+x^2)")

```

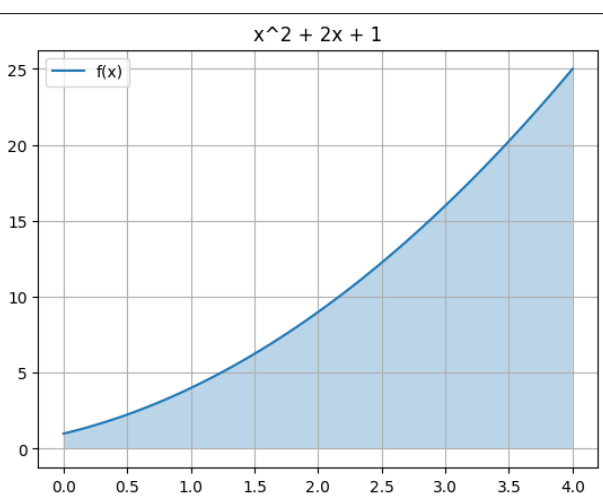
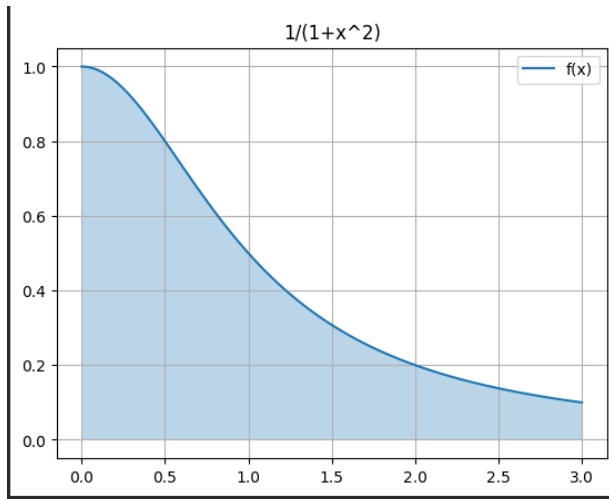
RESULTS & VISUALIZATION



```
*** ---- Integration Results ----
1)  $\int (x^2+2x+1) dx$  from 0 to 4
Trapezoidal: 42.0
Simpson 1/3: 41.33333333333333
Simpson 3/8: n must be multiple of 3 for Simpson 3/8

2)  $\int \ln(1+x) dx$  from 0 to 2
Trapezoidal: 1.2896960448369004
Simpson 1/3: 1.295721475674256
Simpson 3/8: 1.2956013848297627

3)  $\int 1/(1+x^2) dx$  from 0 to 3
Trapezoidal: 1.247811671087533
Simpson 1/3: 1.2470822281167107
Simpson 3/8: 1.2429708222811668
```



Function	Actual Value
$\int (x^2 + 2x + 1) dx$ from 0 to 4	41.3333
$\int \ln(1 + x) dx$ from 0 to 2	1.2958
$\int 1 / (1 + x^2) dx$ from 0 to 3	1.2490

CONCLUSION FOR PART(a)

Numerical Integration:

Simpson's 1/3 Rule produced the most accurate results in all cases. Trapezoidal Rule showed larger error due to linear approximation. Simpson's 3/8 Rule also gave good accuracy but depends on n being multiple of 3.

PART(B)

CODE

```
# -----  
# Modified Euler Method  
# -----  
def modified_euler(f, x0, y0, h, xn):  
    x = [x0]  
    y = [y0]  
  
    while x[-1] < xn:  
        x_n = x[-1]  
        y_n = y[-1]  
  
        y_pred = y_n + h * f(x_n, y_n)  
        y_corr = y_n + (h/2) * (f(x_n, y_n) + f(x_n + h, y_pred))  
  
        x.append(x_n + h)  
        y.append(y_corr)  
  
    return np.array(x), np.array(y)  
  
# -----  
# Runge-Kutta 4th Order  
# -----  
def rk4(f, x0, y0, h, xn):  
    x = [x0]  
    y = [y0]  
  
    while x[-1] < xn:  
        x_n = x[-1]  
        y_n = y[-1]  
  
        k1 = h * f(x_n, y_n)  
        k2 = h * f(x_n + h/2, y_n + k1/2)  
        k3 = h * f(x_n + h/2, y_n + k2/2)  
        k4 = h * f(x_n + h, y_n + k3)  
  
        y_next = y_n + (k1 + 2*k2 + 2*k3 + k4)/6
```

```

        x.append(x_n + h)
        y.append(y_next)

    return np.array(x), np.array(y)

# -----
# Problem 1
# dy/dx = x + y
# Exact: y = 2e^x - x - 1
# -----

f1 = lambda x, y: x + y
exact1 = lambda x: 2*np.exp(x) - x - 1

x_me, y_me = modified_euler(f1, 0, 1, 0.1, 0.2)
x_rk, y_rk = rk4(f1, 0, 1, 0.1, 0.2)

x_exact = np.linspace(0, 0.2, 100)
y_exact = exact1(x_exact)

plt.figure()
plt.plot(x_me, y_me, 'o-', label="Modified Euler")
plt.plot(x_rk, y_rk, 's-', label="RK4")
plt.plot(x_exact, y_exact, label="Exact")
plt.legend()
plt.title("dy/dx = x + y")
plt.grid()
plt.show()

# -----
# Problem 2
# dy/dx = x^2 + y
# -----

f2 = lambda x, y: x**2 + y

```

```

x_me2, y_me2 = modified_euler(f2, 0, 1, 0.1, 0.2)
x_rk2, y_rk2 = rk4(f2, 0, 1, 0.1, 0.2)
exact2 = lambda x: 3*np.exp(x) - x**2 - 2*x - 2

x_exact2 = np.linspace(0, 0.2, 100)
y_exact2 = exact2(x_exact2)

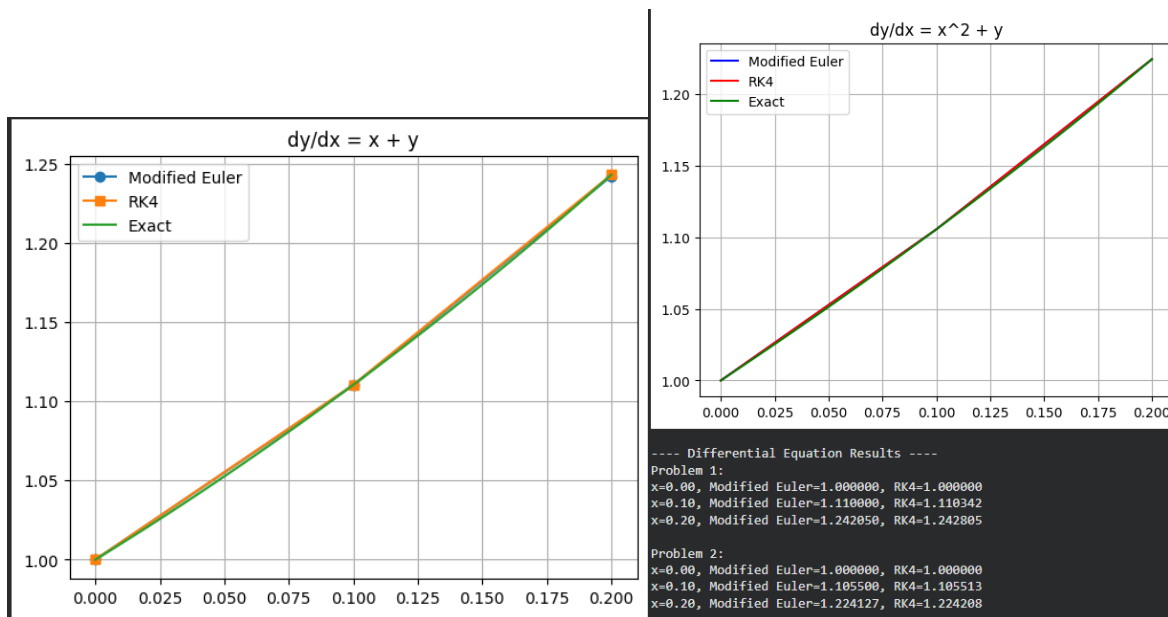
plt.figure()
plt.plot(x_me2, y_me2, 'b-', label="Modified Euler")
plt.plot(x_rk2, y_rk2, 'r-', label="RK4")
plt.plot(x_exact2, y_exact2, 'g-', label="Exact")
plt.legend()
plt.title("dy/dx = x^2 + y")
plt.grid()
plt.show()

# Display values
print("\n---- Differential Equation Results ----")
print("Problem 1:")
for i in range(len(x_me)):
    print(f"x={x_me[i]:.2f}, Modified Euler={y_me[i]:.6f}, RK4={y_rk[i]:.6f}")

print("\nProblem 2:")
for i in range(len(x_me2)):
    print(f"x={x_me2[i]:.2f}, Modified Euler={y_me2[i]:.6f}, RK4={y_rk2[i]:.6f}")

```

RESULTS & VISUALIZATION



 **Problem 1:** $\frac{dy}{dx} = x + y$, exact $y = 2e^x - x - 1$

x	Exact Value
0.00	1.000000
0.10	1.110342
0.20	1.242806

 **Problem 2:** $\frac{dy}{dx} = x^2 + y$, exact $y = 3e^x - x^2 - 2x - 2$

x	Exact Value
0.00	1.000000
0.10	1.105513
0.20	1.224208

CONCLUSION FOR PART(B)

Differential Equations:

RK4 method is significantly more accurate than Modified Euler. Modified Euler improves over simple Euler but still has noticeable deviation. Graphs clearly show RK4 almost overlapping with exact solution.